

PRD for <Product/Feature>

Version History

A PRD ([product requirements document](#)) is a living document that is used throughout development to outline the requirements and features of a product. The document is typically started by a product manager, but is usually contributed to and maintained by a cross-functional development team. It is an essential tool to communicate to stakeholders and guide the development process. For an example, see [Gizmojis PRD](#).

- 1/16/2026 Version 0.1 - Template created
- <date> Version 1.0 - First version
- <date> Version <number> - Update

Related Documents/Links

Use this section to link to other relevant documents, such as key benchmarks, technical papers, market research, summarized brainstorming output, etc.

- Link to research paper #1
- Link to neighboring product
- Link to engineering design
- Link to document 3

Overview

Our triage refinery is a de-bugging co-pilot for autonomy engineers. Its mission is to reduce the time, cognitive load, and repetitive manual effort between the time a failure is detected to create a more informed and productive debugging session. Drop in a failure ticket, get back a reasoned investigation report and a workspace already set up to go. The long term goal is to become the system that transforms messy failure traces into structured engineering memory: from a reusable layer of triaged artifacts, similar case retrieval, and regression knowledge that compounds over time. Think of a digital journal and assistant for debugging complex autonomous systems in non-deterministic environments.

Many engineering teams today utilize many tools for failure investigations: issue trackers, replay viewers and observability tools, simulation systems, dashboards, and internal scripts. However failures often still arrive as fragmented bundles of logs, metadata, links, and notes that must be interpreted and organized before useful debugging can begin. The triage refinery we're envisioning sits on top of existing tools and workflows. It ingests failure logs and tickets and triages it into a structured debugging artifact, retrieves similar past cases and relevant resources, and presents the engineer with the context and maybe recommended next steps (based on similar cases) to start investigation faster. Basically, our goal is to eliminate the setup tax. We want the engineer to come to the debugging session with 100% cognitive capacity, not drained by trying to find the right logs and set up the right tools.

Internally, Triage Refinery uses a multi-agentic pipeline to handle the variable reasoning tasks involved in triage, including evidence extraction, similar-case interpretation, and context-specific recommendation generation.

Our v0 will focus on establishing trust through a co-pilot model. The agent will work transparently with the engineer, showing its reasoning and surfacing uncertainty rather than asserting confident conclusions. Future versions will introduce a multi-agent debate architecture for root cause analysis, and eventually an autonomous mode where the agent can run tests to validate hypotheses without hand-holding. We want the experience to feel like a Sierra AI + Datadog baby for autonomy.

Problem/Benefit

Engineering teams in autonomy and physical AI often have tooling for observing failures, but still face friction in turning raw failure evidence into reusable, structured debugging context. Even when a failure ticket includes a file identifier, timestamp, or replay link, the engineer may still need to interpret the event, understand which subsystem is implicated, compare it to prior failures, gather context from multiple sources, and decide what to inspect first.

The pain points include:

- failure evidence is fragmented across logs, tickets, dashboards, viewers, and historical notes
- similar past incidents and prior fixes are not surfaced consistently
- failure understanding is not normalized into a reusable artifact for future triage
- debugging setup is repetitive and mentally expensive
- future regressions and organization suffer because failure knowledge is not refined and stored consistently, re-doing the same triage work

The product benefit is not that it “solves debugging” automatically. The benefit is that it gives engineers a better starting point and eliminates the setup phase entirely: a structured failure artifact, relevant context, and grounded recommendations that reduce triage friction and improve future reuse.

Opportunity

Identify some opportunities for your product.

What are the opportunities to have an impact in your project about which you are most passionate?

What are realistic targets for those opportunities that would be fulfilling if you achieved them by June?

How niche or how big is the Total Addressable Market (TAM)?

The opportunity is to create a new layer in the autonomy tooling stack: a failure-intelligence and triage orchestration layer that sits between raw failure evidence and the engineer's debugging process.

Existing tools are strong at observability, simulation, replay, or storage, but there is room for a product that turns raw incidents into structured, reusable engineering knowledge.

Opportunities include:

- reducing time from failure detection to investigation-ready state
- creating a canonical failure capsule that teams can search, cluster, compare, and reuse
- improving similar-case retrieval and historical debugging memory
- laying the groundwork for future regression generation and prioritization workflows
- providing a natural-language interface for interacting with complex engineering triage workflows

The opportunity the team is most passionate about is making edge-case discovery and debugging workflows faster through better organization and triage, while preserving a path toward future regression intelligence.

Realistic targets by June:

- demonstrate an end-to-end prototype that ingests a failure ticket and outputs a structured investigation workspace
- validate that the generated failure capsule is useful and understandable
- Show that similar-case retrieval shows relevant history, not just keyword matches
- Confirm the product feels like a useful copilot, not just a superficial dashboard or summary tool.
- show that similar-case retrieval and recommended next steps are meaningfully better than a raw ticket alone

- test whether the product feels like a useful debugging copilot rather than a superficial dashboard

TAM is initially niche if framed narrowly around autonomy and physical AI validation/debugging teams, but potentially expands into robotics, industrial autonomy, embodied AI, simulation-heavy ML systems, and any engineering organization that investigates complex multimodal failures. The near-term addressable market is specialized but high-value; the long-term opportunity is broader if the triage and failure-memory layer proves generalizable.

User segments

Who are you building your product for?

If you have user/customer segments who need certain types of functionality, list them here. User personas/journeys could be described here. Use cases could be another way to think of this as well.

What use cases are you going to nail?

Primary user segments:

1. Autonomy / validation engineers investigating simulation or on-road failures
2. Systems / integration engineers debugging multimodal behavior across logs, viewers, and internal tools
3. Safety / triage / incident review engineers responsible for organizing and escalating failure investigations

Secondary user segments:

4. Engineering managers or leads who want better visibility into failure categories, repeated incidents, and debugging progress
5. ML/perception/planning engineers who want similar-case retrieval and historical context relevant to their subsystem

Core use cases to nail:

- ingest a failure ticket and associated metadata

- generate a concise but structured triage summary
- produce a normalized failure capsule with key extracted fields
- retrieve similar prior incidents from a database
- surface the most relevant resources and next inspection steps
- let an engineer use the system as a first-stop investigation workspace

Initial user journey:

A failure is detected and a ticket is created. The engineer starts up the triage refinery, and an agent parses the failure, retrieves relevant history, generates a structured failure capsule with root cause hypotheses and confidence intervals, and assembles the workspace with pre-loaded tools. The engineer simply reads the report, reviews similar cases, and immediately begins investigating, no setup required.

Value Prop / Differentiators

Think about this from the standpoint of your key features/benefits. Again, you are encouraged to look at the “[Messaging Activation Matrix](#)” and imagine things that you’ll need to have a compelling product relative to what’s out there now.

There should be at least 3 features/benefits you can highlight in your messaging about your product

What is the value prop of your product?

How is this product better than competitor products?

How is this product better than what people are using now?

What are the three most compelling features that will be front and center on press releases, home pages, product packaging, etc.

Value prop

Triage Refinery transforms messy failure evidence into a structured, reusable debugging artifact and investigation workspace. It reduces the effort required to understand what happened, what similar failures have occurred before, and what to inspect next.

Better than competitor products

Unlike observability tools, it is not just about seeing data. Unlike simulation/replay tools, it is not just about reproducing events. Unlike vector databases, it is not just about storing embeddings. Triage Refinery focuses on the refinement step: turning raw incident evidence into structured engineering knowledge and recommended action.

Unlike fixed dashboards or rigid workflows, Triage Refinery uses specialized reasoning components to adapt triage behavior to the specific failure at hand, allowing the system to extract context, interpret relevant prior cases, and prioritize useful next steps.

Better than what people are using now

Current workflows are often fragmented across tickets, dashboards, replay viewers, logs, and memory in people's heads. Triage Refinery centralizes the first phase of investigation into a structured, grounded, reusable triage artifact, without requiring teams to replace their existing tools.

Three most compelling features/benefits

- **Failure Capsule Generation** — Automatically turns a raw failure ticket and related evidence into a structured debugging artifact with normalized fields, summary, and context.
- **Similar-Case Retrieval** — Surfaces the most relevant prior incidents, related failure patterns, and historical context to reduce repeated investigation work.
- **Investigation Copilot Workspace** — Presents recommended next steps, likely relevant resources, and the right context in one place so engineers can start debugging faster.

1.

Functional Requirements

This is the meat of the PRD and describes the solution, features, and functionality. Summarize the high level functionality of the product, and then provide more detail in subsections. This should encompass what you believe are going to be the core features that make your solution valuable.

This explains to various stakeholders how the product should work, why a feature is needed, and how important it is to the overall usefulness of the product. This is what a development team reads when it translates a high level description of the product into visual designs and code.

This could provide some mapping of a feature to which user segment you are building it for. It should also have some prioritization around what must be implemented or could be dropped if time runs out.

Functionality	User segment 1	User segment 2	User story	Priority
Failure Ingestion and Parsing	Yes	Yes	As an engineer, I want to input a failure ticket and evidence so the system can begin triage automatically.	Must have
Failure capsule generation	Yes	Yes	As an engineer, I want a structured summary of what failed, key conditions, likely subsystem, and important metadata so I can orient quickly.	Must have
Triage Agent with Retrieval	Yes	Yes	As an engineer, I want the agent to retrieve similar past failures and explain why they match, so I can avoid red-doing investigation work	Must have
Pre-loaded investigation workspace			As an engineer, I want my tools already at the right place at the right time so I can start debugging immediately without manual setup.	Must Have
Recommendation Engine			As an engineer, I want grounded next-step suggestions so I know what to inspect first.	Nice Have
Multi-Agent Debate for root cause analysis	Yes	Yes	As an engineer I want competing hypotheses stress-tested against each other with independent evidence retrieval, so I get a more robust reasoning towards root-cause conclusion	Nice Have

Feature 1

Description of feature 1

Failure ingestion and parsing

The system accepts a failure ticket and related metadata as input. This can include ticket title, description, scenario ID, timestamp, tags, subsystem hints, and attached or referenced artifacts. This solves the problem of fragmented entry points by giving the system a consistent starting point for

triage. This feature is important because every downstream function depends on clean ingestion and normalization of raw evidence. Priority: highest. Needed immediately for V0.

Feature 1.1

More detailed description of functionality within feature 1

- accept mock ticket (JIRA, slack, etc) input via API or UI
 - Source_type
 - Source_ref
 - Title
 - raw_text
 - Event_timestamp
 - Agent_id
 - Scenario_id
 - Artifacts_ref
- extract structured fields from raw text and metadata
- normalize fields into an internal schema
- store original input and normalized representation for downstream retrieval

Feature 2

Description of feature 2

Failure capsule generation

The system generates a structured triage artifact, or failure capsule, from ingested evidence. The failure capsule contains a concise summary plus normalized attributes such as scenario type, failure mode hints, likely subsystem, severity cues, key timestamps, tags, and related evidence references. This solves the problem of raw tickets being inconsistent and not reusable as engineering memory. Priority: highest. Needed immediately for V0.

Feature 2.1

More detailed description of functionality within feature 2

- generate short natural-language triage summary
- extract canonical fields from ticket and metadata

- label failure taxonomy where possible
- store the capsule for future search and retrieval

Feature 3

Description of feature 3

The Triage Agent is the core of the product. It takes the normalized failure capsule and orchestrates three sub-tasks: retrieval of similar historical cases, generation of root cause hypotheses, and assembly of the investigation workspace. The agent's behavior adapts to the failure type — different failures require different retrieval strategies, different subsystem focus, and different next-step recommendations.

The agent operates in copilot mode for V0: it surfaces its reasoning at each step and surfaces uncertainty explicitly rather than asserting false confidence. Engineers can read the agent's chain of reasoning, disagree with a specific step, and override conclusions before acting.

- **Feature 3.1 — Similar-case retrieval**
 - Embed normalized failure capsule and query vector DB of historical capsules
 - Rank results by semantic similarity across structured fields and narrative
 - For each retrieved case, surface the similarity reason — what dimensions matched
 - Include prior resolution or outcome if available
 - Flag low-confidence retrieval explicitly rather than silently returning weak matches
- **Feature 3.2 — Root cause hypothesis generation**
 - Generate 2–3 distinct root cause hypotheses from ticket evidence and retrieved cases
 - Assign a confidence level to each hypothesis (high / medium / low / uncertain)
 - Cite the specific evidence supporting each hypothesis — no ungrounded conclusions
 - Surface what additional signal would raise or lower confidence
 - Include an explicit “insufficient evidence” option when the data genuinely does not support a conclusion
- **Feature 3.3 — Workspace assembly**
 - Identify which tools are relevant to this failure type (replay viewer, log explorer, simulation runner, etc.)
 - Pre-configure and sync tools to the failure context: timestamp, scenario ID, subsystem, relevant log filters
 - Surface relevant internal resource links and documentation

- Present recommended first inspection steps tied to the top hypothesis

Feature 4

Description of feature 4

Investigation workspace

The system presents the failure capsule, similar cases, relevant artifacts, and resource links in one interface. This solves the workflow problem of switching between fragmented sources before productive debugging can begin. Priority: highest. Needed in V0 as a minimal UI shell.

Feature 4.1

More detailed description of functionality within feature 4

- show ticket context and triage summary
- show structured failure capsule
- show similar incidents
- show relevant links/resources/tools
- support simple engineer notes or feedback on usefulness

Feature 5

Description of feature 5

Grounded recommendation engine

The system recommends the most useful next steps based on the triaged context and retrieved similar incidents. This solves the problem of engineers spending time deciding what to inspect first. Priority: should have for V0, but can be simplified if time runs out.

Feature 5.1

More detailed description of functionality within feature 5

- generate recommended inspection steps
- tie recommendations to retrieved evidence or prior incidents
- suggest likely subsystem or owner only when grounded
- avoid generic ungrounded advice

Feature 6

Description of feature 6

Multi-Agent Debate

In V1, the single Triage Agent is replaced with a three-agent debate architecture designed to stress-test root cause hypotheses before presenting conclusions to the engineer. This architecture directly addresses the core reliability risk of single-agent reasoning: a single agent will confidently pursue the most probable explanation without adequately considering alternatives.

Architecture:

- Proposal Agent: independently retrieves supporting evidence and argues for each root cause hypothesis
- Critic Agent: independently retrieves disconfirming evidence (blind to the Proposal's queries and results) and argues against each hypothesis
- Judge Agent: receives both agents' full evidence sets and debate transcripts, reconciles conflicting evidence, and produces a final verdict with confidence and explicit reasoning

Critical design constraint: the Critic Agent performs its retrieval before seeing the Proposal's conclusions. This prevents the contamination failure mode where the Critic reverse-engineers counterarguments from knowing the opposing position — which would produce motivated retrieval rather than genuine disconfirmation.

The debate transcript is shown in the workspace, not just the verdict. Engineers can read the full Proposal-Critic exchange and disagree with specific reasoning steps. Every engineer override becomes a training signal for agent improvement.

Complete above this line for the initial PRD revision due in the first part of the course. Below the line you should fill out as your product and plan evolves and have in a fully completed state by the end of the course

Go to Market Plan

A lot of activity happens during development to define your product. Building a great product doesn't mean users will discover and use it. Identify all the messaging mediums you will use and the content you will communicate through them.

How will you educate users about your product? How will you learn from them?

How do you plan to launch this feature with your target users?

How will users learn how to use it?

Link or embed your [Messaging Activation Matrix](#)

Metrics for success

This section is used to identify important metrics to measure product performance. How do you measure success and adoption for the product? These can be internal or external metrics, adoption metrics, performance metrics, logging, survey data, and more. List concrete metrics to track here.

Objective Function

What is the objective function? What do you want your product to achieve and how will you measure it? Be clear about what the objective is and how that is measured.

Goals

What numbers do you aim for? What would be initial targets? What is successful? What is aspirational?

Timeline

Major milestones

Design considerations

Any high level UX/UI considerations to mention, but primarily link to any UX research or any prototypes as things progress.

Technical considerations

Anything that's relevant for/from engineering, such as key technical challenges or risks. Platforms supported.

Are there any product assumptions/features that impact engineering design/cost? One very core example is in what medium does the product need to be delivered? Mobile (iOS, Android, both?), Web, custom hardware, commodity hardware, etc.

Open questions (optional)

Log any open questions here that you have. For example:

- What decisions still need to be made?
- What research still needs to be conducted?
- What does product user/market fit look like?
- What is the MVP?
- What are you trying to learn in each phase?